



Project Title - Seasonal_Agriculture_Performance_Analysis

Student Name : KANISHKARAJ R P R

College Name : MISRIMAL NAVAJEE MUNOTH JAIN ENGINEERING COLLEGE

AICTE STU ID : STU684d497ca665e1749895548



PROBLEM STATEMENT

- Agricultural activities are influenced by seasonal variations in environmental conditions, farming practices, resource availability and market conditions.
- As a result, agricultural performance may differ from one season to another. However, raw agricultural data does not clearly explain how agricultural performance changes across seasons or what patterns can be observed in different seasonal conditions.
- The problem is to analyze the given agricultural dataset and investigate seasonal differences in agricultural performance by identifying meaningful patterns, trends, relationships and variations within the available data.



Project Description

- **Comprehensive Data Scope:** Analyzed 4,000 farm records across 28 parameters to investigate agricultural performance and resource utilization across Kharif, Rabi, and Zaid seasons.
- **Rigorous Data Governance:** Addressed 120 missing values using deterministic area-production yield recovery and context-aware grouped median imputation, ensuring 0% data loss.
- **Multi-Dimensional Analytics:** Executed univariate, bivariate, multivariate, and correlation models to map complex dependencies between soil chemistry, water efficiency, pest risk, and financial margins.
- **Actionable Operational Inquiries:** Implemented three targeted analytical inquiries to evaluate drip irrigation ROI, chemical fertilizer diminishing returns, and regional crop profitability matrices.
- **Strategic Impact:** Delivered evidence-based recommendations for sustainable agricultural planning and resource optimization without reliance on complex machine learning.

WHO ARE THE END USERS?

- **Agricultural Planners & Policymakers:** To design data-driven resource subsidies, optimize water distribution networks, and establish sustainable agricultural zoning policies.
- **Agribusinesses & Supply Chain Managers:** To forecast seasonal harvest volumes, mitigate supply chain bottlenecks, and anticipate regional commodity pricing trends.
- **Farmers & Agronomists:** To optimize chemical fertilizer application rates, reduce input cost erosion, and adopt water-efficient drip irrigation methods for superior farm profitability.

Technology Used

- **Programming Language:** Python
- **Development Environment:** Google Colab
- **Core Data Analytics & Visualization Libraries:**

1. Pandas & NumPy - Utilized for data ingestion, cleaning, primary key deduplication, missing value imputation, and quantitative aggregation.

2. Matplotlib & Seaborn - Utilized for generating exploratory visualizations, including univariate distribution histograms, bivariate cost-revenue evaluations, multivariate pest risk boxplots, and correlation heatmaps.



RESULTS

```
 # Import required libraries

import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

%matplotlib inline

sns.set_theme(style="whitegrid")
pd.set_option("display.max_columns", None)
pd.set_option("display.float_format", lambda x: f"{x:,.2f}")
```

RESULTS



Load the dataset

```
file_path="/seasonal_agriculture_performance_dataset.csv"  
df= pd.read_csv(file_path)  
print("Dataset loaded successfully")
```

... Dataset loaded successfully

Dataset shape

```
print("Number of rows", df.shape[0])  
print("Number of columns", df.shape[1])
```

```
Number of rows 4000  
Number of columns 28
```



RESULTS

Top 5 rows

df.head(5)

	Farm_ID	State	District	Crop	Season	Farm_Area_Hectares	Rainfall_mm	Avg_Temperature_C	Humidity_pct	Sunlight_Hours_Day	Soil_pH	Soil_Moi
0	SF10001	Andhra Pradesh	Rajkot	Wheat	Kharif	0.53	486.40	24.30	69.10	5.80	7.14	
1	SF10002	Maharashtra	Nalgonda	Maize	Kharif	6.53	855.40	25.70	78.40	5.10	5.20	
2	SF10003	Telangana	Warangal	Pulses	Rabi	4.86	455.60	23.90	56.40	10.30	5.82	
3	SF10004	Telangana	Indore	Rice	Kharif	4.50	753.20	29.70	65.20	6.30	5.98	
4	SF10005	Karnataka	Ludhiana	Maize	Zaid	4.21	101.60	34.10	55.20	6.50	6.90	

RESULTS

Bottom 5 rows

df.tail(5)

	Farm_ID	State	District	Crop	Season	Farm_Area_Hectares	Rainfall_mm	Avg_Temperature_C	Humidity_pct	Sunlight_Hours_Day	Soil_pH	Soil_Mo
3995	SF13996	Karnataka	Rajkot	Wheat	Kharif	4.72	865.40	27.30	51.40	6.30	7.97	
3996	SF13997	Karnataka	Krishna	Chilli	Kharif	8.77	947.50	29.20	85.50	6.10	6.57	
3997	SF13998	Andhra Pradesh	Erode	Pulses	Zaid	10.47	80.00	35.40	68.50	8.60	7.25	
3998	SF13999	Punjab	Warangal	Cotton	Kharif	1.73	475.00	28.20	61.70	6.40	6.11	
3999	SF14000	Punjab	Nashik	Wheat	Kharif	3.31	984.30	25.80	77.20	5.90	6.16	



RESULTS

```
# Random samples of records
```

```
df.sample(5, random_state=42)
```

	Farm_ID	State	District	Crop	Season	Farm_Area_Hectares	Rainfall_mm	Avg_Temperature_C	Humidity_pct	Sunlight_Hours_Day	Soil_pH	Soil_I
555	SF10556	Andhra Pradesh	Nashik	Chilli	Rabi	7.94	456.80	25.30	47.70	7.60	7.10	
3491	SF13492	Punjab	Warangal	Wheat	Kharif	12.44	601.90	27.20	60.20	8.30	7.66	
527	SF10528	Maharashtra	Indore	Cotton	Kharif	6.35	1,213.70	26.30	73.60	7.10	6.33	
3925	SF13926	Madhya Pradesh	Nashik	Wheat	Rabi	12.52	374.00	17.90	59.70	9.40	7.57	
2989	SF12990	Gujarat	Rajkot	Chilli	Rabi	13.98	197.80	19.00	55.80	9.20	7.31	

RESULTS

```
# Column names
```

```
print(df.columns.tolist())
```

```
['Farm_ID', 'State', 'District', 'Crop', 'Season', 'Farm_Area_Hectares', 'Rainfall_mm', 'Avg_Temperature_C', 'Humidity_pct', 'Sunlight_Hours_Day', 'Soil']
```

```
#Data types and non-null counts
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 4000 entries, 0 to 3999
```

```
Data columns (total 28 columns):
```

#	Column	Non-Null Count	Dtype
0	Farm_ID	4000 non-null	object
1	State	4000 non-null	object
2	District	4000 non-null	object
3	Crop	4000 non-null	object
4	Season	4000 non-null	object
5	Farm_Area_Hectares	4000 non-null	float64
6	Rainfall_mm	3952 non-null	float64

RESULTS

```
# Numerical summary statistics
```

```
df.describe()
```

	Farm_Area_Hectares	Rainfall_mm	Avg_Temperature_C	Humidity_pct	Sunlight_Hours_Day	Soil_pH	Soil_Moisture_pct	Nitrogen_kg_ha	Phosphorus_kg_ha
count	4,000.00	3,952.00	4,000.00	4,000.00	4,000.00	4,000.00	3,960.00	4,000.00	4,000.00
mean	7.89	601.15	26.82	63.21	7.32	6.70	26.51	120.25	57.78
std	4.22	288.93	3.82	11.96	1.12	0.64	6.71	31.22	17.02
min	0.50	80.00	16.20	25.00	3.50	5.20	8.00	40.00	15.00
25%	4.26	370.20	23.90	54.77	6.60	6.26	21.70	98.50	46.20
50%	7.89	582.00	26.90	63.00	7.30	6.69	26.40	120.10	57.50
75%	11.65	834.95	29.60	71.90	8.10	7.14	31.30	141.22	69.40
max	15.00	1,395.20	39.70	95.00	11.00	8.40	47.00	220.00	120.00

RESULTS

```
# Categorical summary  
  
df.describe(include="object")
```

	Farm_ID	State	District	Crop	Season	Irrigation_Method
count	4000	4000	4000	4000	4000	4000
unique	4000	8	10	8	3	4
top	SF13984	Andhra Pradesh	Ludhiana	Rice	Kharif	Flood
freq	1	529	427	690	1779	1310



RESULTS

```
# MISSING VALUES IDENTIFICATION

import warnings
warnings.filterwarnings('ignore')

# Audit missing values from current dataset
missing_counts = df.isnull().sum()
missing_percent = (missing_counts / len(df)) * 100

missing_df = pd.DataFrame({
    'Missing_Count': missing_counts,
    'Percentage_%': missing_percent
})

# Display only columns that contain missing values
missing_identified = missing_df[missing_df['Missing_Count'] > 0]
print("--- Identified Missing Values in Dataset ---")
display(missing_identified)
```

... --- Identified Missing Values in Dataset ---

	Missing_Count	Percentage_%
Rainfall_mm	48	1.20
Soil_Moisture_pct	40	1.00



RESULTS

```
# MISSING VALUES HANDLING

# 1. Deterministic Recovery: Calculate exact Yield from Production and Farm Area
df["Yield_Tonnes_Ha"] = df["Yield_Tonnes_Ha"].fillna(
    df["Production_Tonnes"] / df["Farm_Area_Hectares"]
)

# 2. Climate-based Imputation: Group-wise median by State and Season
df["Rainfall_mm"] = df["Rainfall_mm"].fillna(
    df.groupby(["State", "Season"])["Rainfall_mm"].transform("median")
)

# 3. Moisture Imputation: Group-wise median by Irrigation Method and Season
df["Soil_Moisture_pct"] = df["Soil_Moisture_pct"].fillna(
    df.groupby(["Irrigation_Method", "Season"])["Soil_Moisture_pct"].transform("median")
)

# Verification: Confirm zero missing values remain
print(f"Total remaining missing values: {df.isnull().sum().sum()}")
```

Total remaining missing values: 0

RESULTS

```
# DUPLICATE RECORDS AUDIT & HANDLING

# Identification
total_duplicates = df.duplicated().sum()
farm_id_duplicates = df["Farm_ID"].duplicated().sum()

print("--- Duplicate Record Audit ---")
print(f"Total duplicate rows: {total_duplicates}")
print(f"Duplicate Farm_IDs: {farm_id_duplicates}")

*** --- Duplicate Record Audit ---
Total duplicate rows: 0
Duplicate Farm_IDs: 0
```

RESULTS

```
# Handling: Drop duplicate records if any exist
initial_rows = len(df)
df = df.drop_duplicates(subset=["Farm_ID"]).reset_index(drop=True)
print(f"Deduplication complete: {initial_rows - len(df)} duplicate records removed.")
print(f"Clean Dataset Shape: {df.shape}")
```

```
... Deduplication complete: 0 duplicate records removed.
Clean Dataset Shape: (4000, 28)
```

```
# Checking datatypes again
```

```
df.dtypes
```

```
...
                                0
Farm_ID                        object
State                          object
District                       object
Crop                           object
Season                         object
```

RESULTS

```
# OUTLIERS INVESTIGATION

# 1. Statistical Outlier Audit via 1.5 * IQR Rule
numeric_cols = df.select_dtypes(include=[np.number]).columns.tolist()

outlier_records = []
for col in numeric_cols:
    q25 = df[col].quantile(0.25)
    q75 = df[col].quantile(0.75)
    iqr = q75 - q25
    lower_lim = q25 - 1.5 * iqr
    upper_lim = q75 + 1.5 * iqr

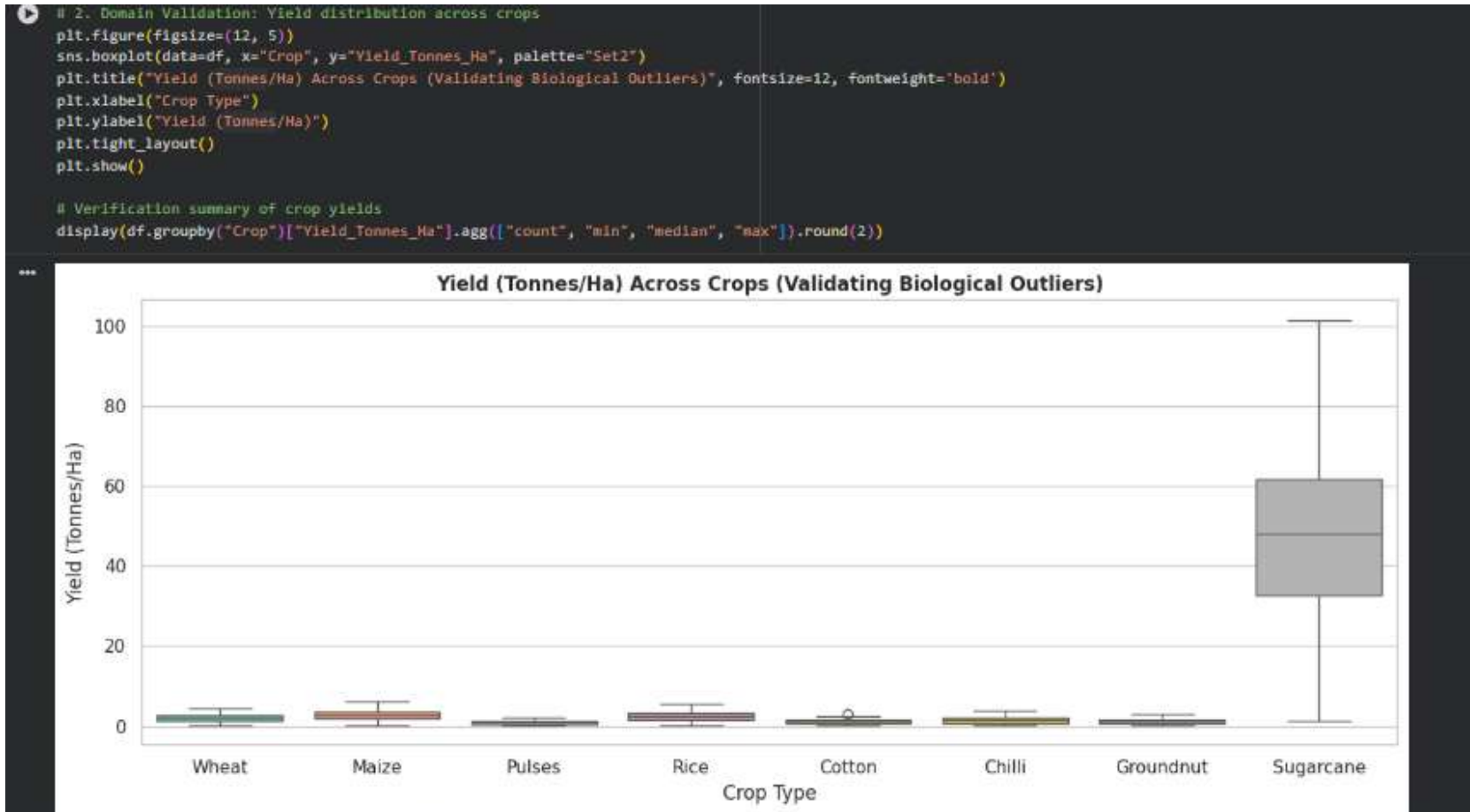
    outlier_cnt = ((df[col] < lower_lim) | (df[col] > upper_lim)).sum()
    if outlier_cnt > 0:
        outlier_records.append({
            "Feature": col,
            "Outlier_Count": outlier_cnt,
            "Outlier_Share_%": round((outlier_cnt / len(df)) * 100, 2),
            "Lower_Bound": round(lower_lim, 2),
            "Upper_Bound": round(upper_lim, 2),
            "Min_Val": round(df[col].min(), 2),
            "Max_Val": round(df[col].max(), 2)
        })

outlier_report = pd.DataFrame(outlier_records)
print("--- Statistical Outlier Report ---")
display(outlier_report)
```

*** --- Statistical Outlier Report ---

	Feature	Outlier_Count	Outlier_Share_%	Lower_Bound	Upper_Bound	Min_Val	Max_Val
0	Avg_Temperature_C	2	0.05	15.35	38.15	18.20	39.70
1	Humidity_pct	6	0.15	29.09	97.59	25.00	95.00
2	Sunlight_Hours_Day	26	0.65	4.35	10.35	3.50	11.00
3	Soil_Moisture_pct	3	0.08	7.30	45.70	8.00	47.00

RESULTS



RESULTS

```
# UNIVARIATE ANALYSIS

fig, axes = plt.subplots(2, 2, figsize=(14, 10))

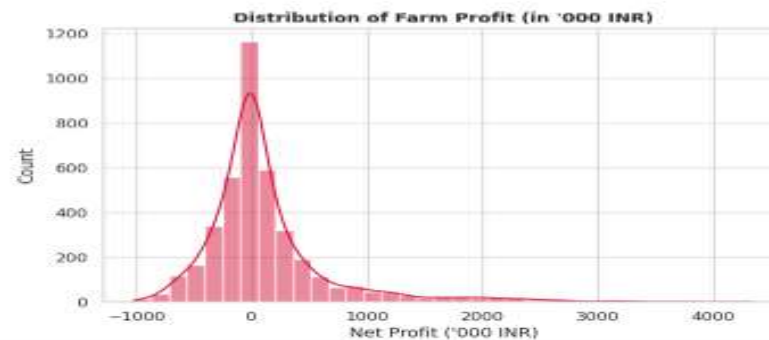
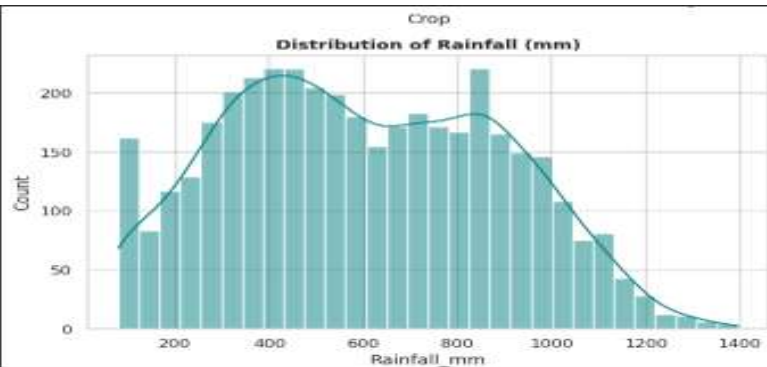
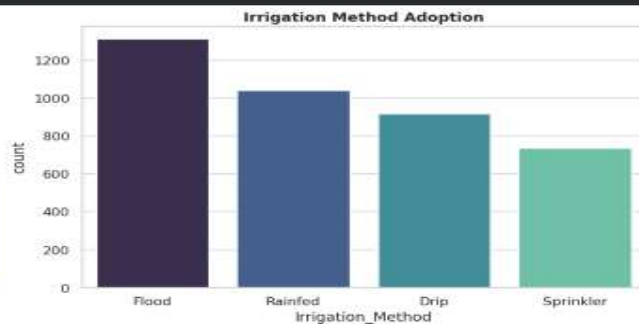
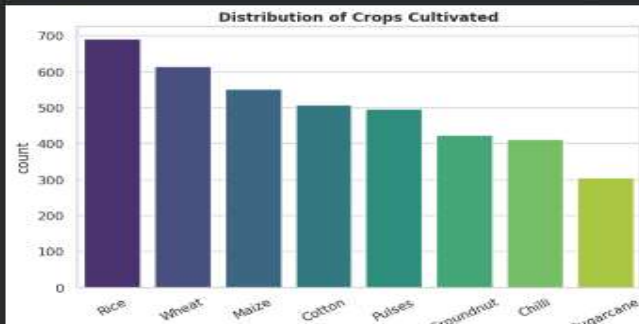
# 1. Crop Distribution
sns.countplot(data=df, x="Crop", order=df["Crop"].value_counts().index, palette="viridis", ax=axes[0, 0])
axes[0, 0].set_title("Distribution of Crops Cultivated", fontsize=12, fontweight='bold')
axes[0, 0].tick_params(axis='x', rotation=30)

# 2. Irrigation Method Distribution
sns.countplot(data=df, x="Irrigation_Method", order=df["Irrigation_Method"].value_counts().index, palette="mako", ax=axes[0, 1])
axes[0, 1].set_title("Irrigation Method Adoption", fontsize=12, fontweight='bold')

# 3. Rainfall Distribution
sns.histplot(df["Rainfall_mm"], kde=True, hist=True, color="teal", ax=axes[1, 0])
axes[1, 0].set_title("Distribution of Rainfall (mm)", fontsize=12, fontweight='bold')

# 4. Net Profit Distribution
sns.histplot(df["Profit_INR"] / 1000, kde=True, hist=True, color="crimson", ax=axes[1, 1])
axes[1, 1].set_title("Distribution of Farm Profit (in '000 INR)", fontsize=12, fontweight='bold')
axes[1, 1].set_xlabel("Net Profit ('000 INR)")

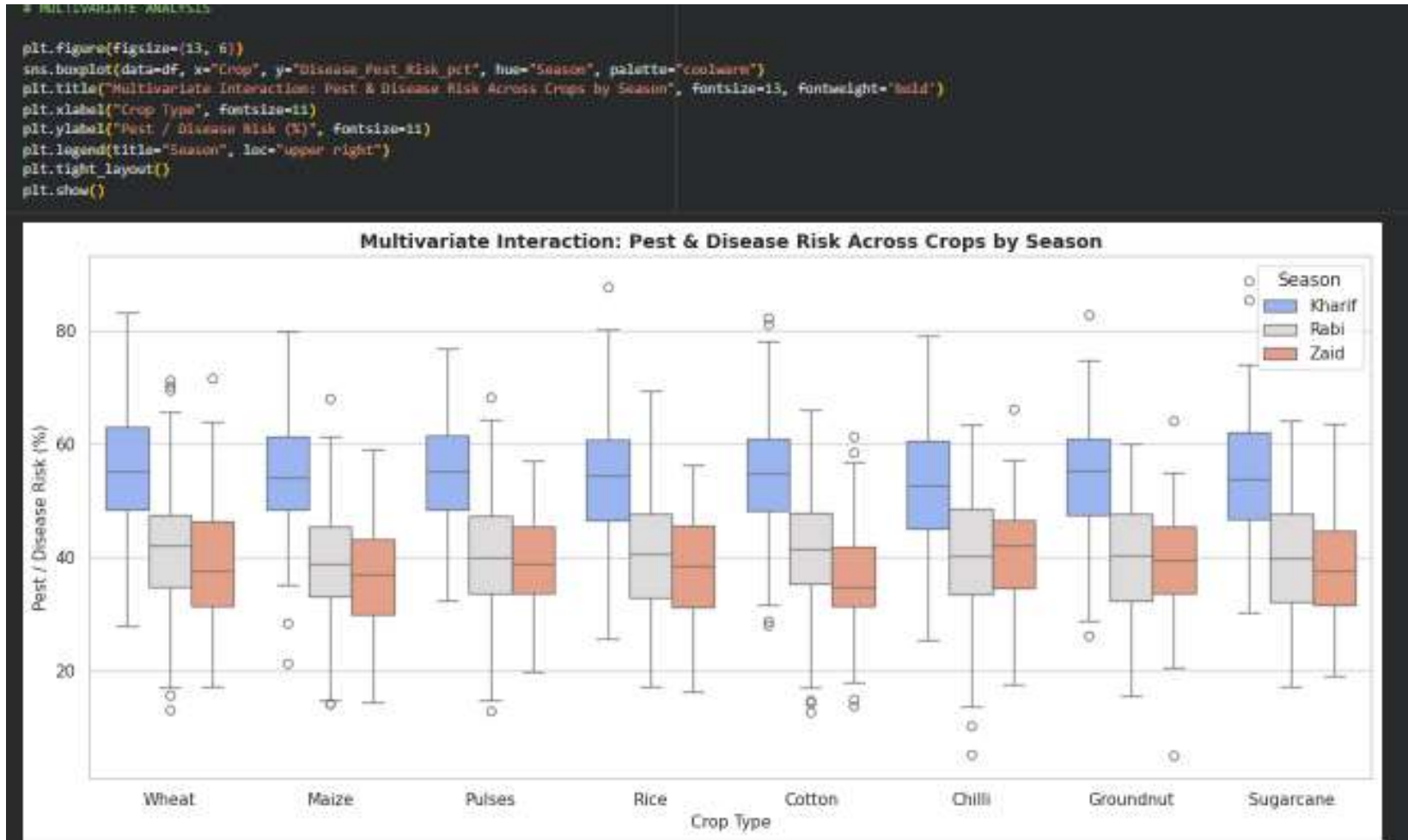
plt.tight_layout()
plt.show()
```



RESULTS



RESULTS

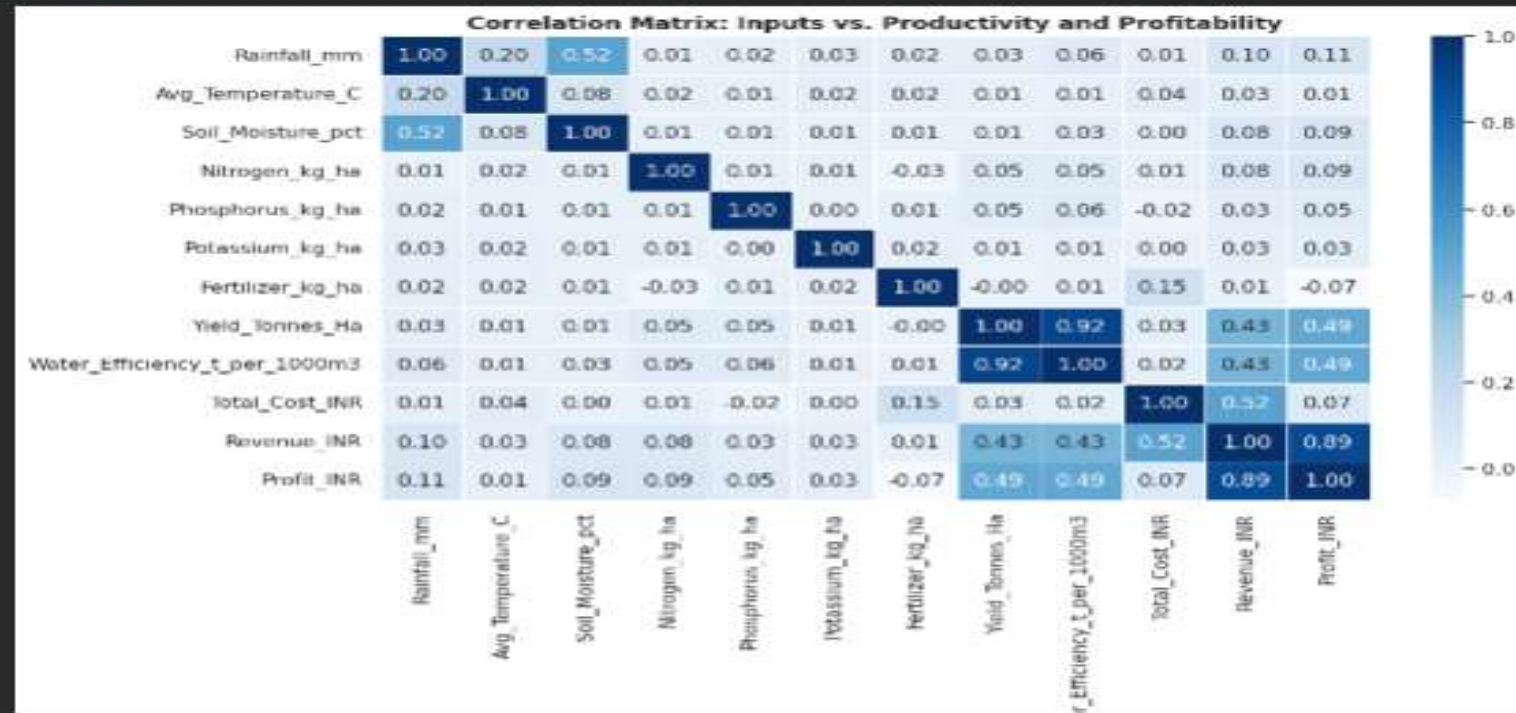


RESULTS

```
# CORRELATION ANALYSIS
```

```
corr_features = [  
    "Rainfall_mm", "Avg_Temperature_C", "Soil_Moisture_pct",  
    "Nitrogen_kg_ha", "Phosphorus_kg_ha", "Potassium_kg_ha",  
    "Fertilizer_kg_ha", "Yield_Tonnes_Ha", "Water_Efficiency_t_per_1000m3",  
    "Total_Cost_INR", "Revenue_INR", "Profit_INR"  
]
```

```
plt.figure(figsize=(12, 8))  
sns.heatmap(df[corr_features].corr(), annot=True, fmt=".2f", cmap="Blues", linewidths=0.5)  
plt.title("Correlation Matrix: Inputs vs. Productivity and Profitability", fontsize=15, fontweight='bold')  
plt.tight_layout()  
plt.show()
```



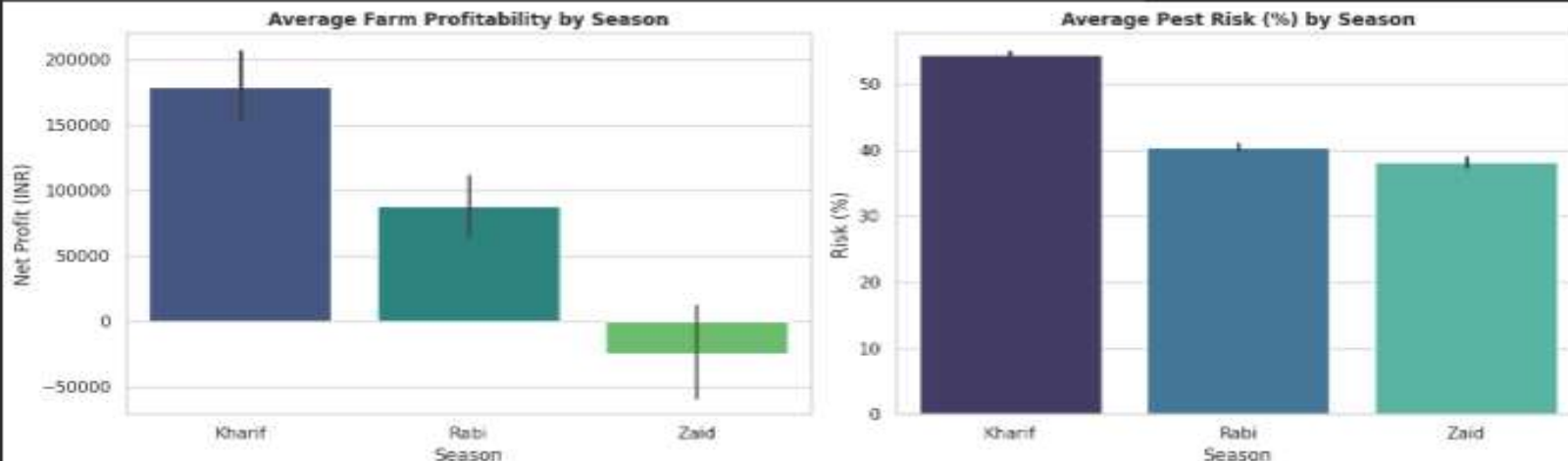
RESULTS

SEASONAL COMPARISONS

```
seasonal_perf = df.groupby("Season").agg(  
    Yield_Avg=("Yield_Tonnes_Ha", "mean"),  
    Rainfall_Avg=("Rainfall_mm", "mean"),  
    Temperature_Avg=("Avg_Temperature_C", "mean"),  
    Pest_Risk_Avg=("Disease_Pest_Risk_pct", "mean"),  
    Water_Efficiency_Avg=("Water_Efficiency_t_per_100bell", "mean"),  
    Total_Cost_Avg=("Total_Cost_INR", "mean"),  
    Revenue_Avg=("Revenue_INR", "mean"),  
    Profit_Avg=("Profit_INR", "mean")  
).reset_index()  
  
print("--- Seasonal Comparison Summary ---")  
display(seasonal_perf.round(2))  
  
fig, axes = plt.subplots(1, 2, figsize=(14, 5))  
sns.barplot(data=df, x="Season", y="Profit_INR", palette="viridis", ax=axes[0])  
axes[0].set_title("Average Farm Profitability by Season", fontsize=12, fontweight="bold")  
axes[0].set_ylabel("Net Profit (INR)")  
  
sns.barplot(data=df, x="Season", y="Disease_Pest_Risk_pct", palette="mako", ax=axes[1])  
axes[1].set_title("Average Pest Risk (%) by Season", fontsize=12, fontweight="bold")  
axes[1].set_ylabel("Risk (%)")  
  
plt.tight_layout()  
plt.show()
```

--- Seasonal Comparison Summary ---

	Season	Yield_Avg	Rainfall_Avg	Temperature_Avg	Pest_Risk_Avg	Water_Efficiency_Avg	Total_Cost_Avg	Revenue_Avg	Profit_Avg
0	Kharif	5.63	852.11	28.45	54.47	5.88	531,804.41	710,719.06	178,914.65
1	Rabi	5.09	435.93	23.49	40.48	5.19	513,836.58	601,526.05	87,689.47
2	Zaid	4.63	299.34	31.04	38.22	4.41	543,976.73	519,171.90	-24,804.82



RESULTS

3 STUDENT-DESIGNED ANALYSES

Inquiry 1: Drip vs. Traditional Irrigation Efficiency & Profitability

```
irr_audit = df.groupby("Irrigation_Method").agg(  
    Water_Used_m3=("Water_Used_m3", "mean"),  
    Water_Efficiency=("Water_Efficiency_L_per_1000m3", "mean"),  
    Yield_Tonnes_Ha=("Yield_Tonnes_Ha", "mean"),  
    Profit_INR=("Profit_INR", "mean")  
).reset_index()
```

```
print("--- Inquiry 1: Irrigation Performance Audit ---")  
display(irr_audit.round(2))
```

Inquiry 2: Law of Diminishing Returns Across Fertilizer Tiers

```
df["Fertilizer_Tier"] = pd.qcut(df["Fertilizer_kg_ha"], q=3, labels=["Low", "Medium", "High"])  
fert_audit = df.groupby("Fertilizer_Tier").agg(  
    Fertilizer_kg_ha=("Fertilizer_kg_ha", "mean"),  
    Yield_Tonnes_Ha=("Yield_Tonnes_Ha", "mean"),  
    Total_Cost_INR=("Total_Cost_INR", "mean"),  
    Profit_INR=("Profit_INR", "mean")  
).reset_index()
```

```
plt.figure(figsize=(8, 4))  
sns.barplot(data=df, x="Fertilizer_Tier", y="Profit_INR", palette="Blues")  
plt.title("Inquiry 2: Profit Erosion from High Fertilizer Application", fontsize=12, fontweight='bold')  
plt.ylabel("Average Profit (INR)")  
plt.show()  
display(fert_audit.round(2))
```

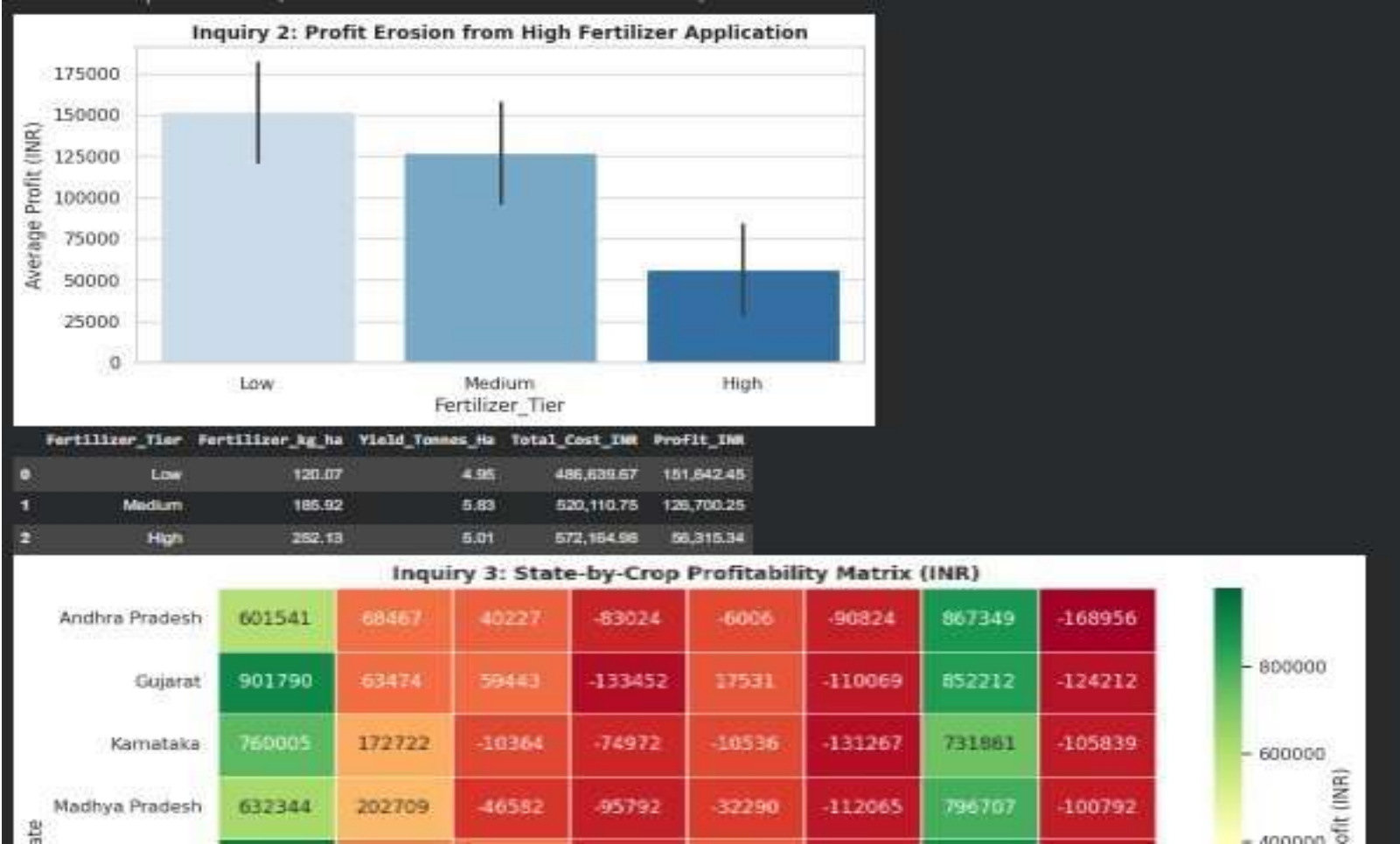
Inquiry 3: Regional Commercial Crop Profitability Matrix

```
state_crop_matrix = df.pivot_table(index="State", columns="Crop", values="Profit_INR", aggfunc="mean")  
plt.figure(figsize=(11, 5))  
sns.heatmap(state_crop_matrix, cmap="magma", annot=True, fmt=".0f", linewidths=0.5, cbar_kws={"label": "Mean Profit (INR)"})  
plt.title("Inquiry 3: State-by-Crop Profitability Matrix (INR)", fontsize=13, fontweight='bold')  
plt.tight_layout()  
plt.show()
```

--- Inquiry 1: Irrigation Performance Audit ---

	Irrigation_Method	Water_Used_m3	Water_Efficiency	Yield_Tonnes_Ha	Profit_INR
0	Drip	5,918.75	6.27	6.68	219,626.00
1	Flood	8,026.47	3.44	4.86	73,354.02
2	Rainfed	3,549.55	7.96	4.60	79,050.37
3	Sprinkler	6,208.26	4.67	5.16	91,121.08

RESULTS



Future scope

- **Predictive Yield Modeling:** Integrate real-time IoT soil sensors and live meteorological forecasting APIs to build machine learning models that forecast crop yields ahead of harvest.
- **Dynamic Market Price Tracking:** Incorporate macro-level market pricing feeds and daily mandi price fluctuations to evaluate real-time revenue and profit volatility.
- **Automated Pest & Disease Early Warning:** Expand the current analytics pipeline into an automated classification framework that uses environmental humidity and temperature triggers to predict pest outbreaks before crop damage occurs.
- **Precision Farming Mobile Integration:** Translate the data insights into a lightweight mobile dashboard for local farmers to receive real-time, localized recommendations on optimal irrigation schedules and fertilizer thresholds.

GitHub Link

https://github.com/RPRK01/Seasonal_Agriculture_Performance_Analysis

VOIS Course completion certificate -

Course name- Data Visualization



Thank you